

Risks and Trade-offs

Risks

Risk 1: Redis Counter Loss on Restart

Description. ShortCodeGenerator holds a batch of up to 1000 pre-allocated IDs in JVM memory. On restart or crash, the unused portion is lost. The Redis counter advances by 1000 on each batch claim, so counter gaps up to 999 can appear.

Impact. Gaps in the counter sequence. No duplicates, just unused numbers.

Mitigation. The PostgreSQL UNIQUE constraint on code is the authoritative guarantee here. Gaps are cosmetically imperfect, not harmful. A system restarting 100 times a day wastes at most 100,000 counter values, and with Base62, 6-character codes support 62^6 , about 56 billion values, so the space isn't exhausted for decades even under heavy restarts. If gap-free sequences were required for audit or compliance reasons, batch size could drop to 1 at the cost of one Redis call per creation.

Residual risk. Low.

Risk 2: Kafka Consumer Lag

Description. Click events are processed asynchronously. If the consumer falls behind, due to DB slowness or high volume, there's a delay between a click and its appearance in analytics.

Impact. Analytics are eventually consistent, not real-time. Under high load, lag could be minutes.

Mitigation. Analytics are off the critical path, so redirects are unaffected by lag. Kafka retains messages for 7 days by default, so a prolonged consumer outage doesn't lose events, only delays them. The 3 partitions on `click-events` allow up to 3 concurrent consumers.

Residual risk. Low for redirect reliability, medium for analytics freshness under load.

Risk 3: PostgreSQL Single Point of Failure

Description. This prototype runs a single PostgreSQL instance. If it fails, both URL creation and, on a cache miss, redirects fail.

Mitigation (in scope). A Docker health check gates app startup on Postgres readiness, and `ddl-auto: update` recovers the schema automatically on restart.

Mitigation (out of scope, production). Streaming replication with automated failover, Patroni or RDS Multi-AZ. For redirects specifically, a warm Redis cache limits exposure to cache-miss failures, and a read replica could serve the DB fallback.

Residual risk. High for the prototype. Documented limitation.

Risk 4: In-Memory Rate Limiting

Description. Rate-limit buckets live in a `ConcurrentHashMap`. They reset on restart and are independent per instance when there are multiple app instances.

Impact. A restart grants everyone a fresh burst quota. A client aware of N instances can round-robin for N times 10 requests per minute.

Mitigation (in scope). Limits apply only to URL creation, not redirects, which is the abuse surface that actually matters. Restart frequency is low in a healthy system anyway.

Mitigation (production). bucket4j-redis for shared bucket state across instances, or push rate limiting up to an API gateway layer like Kong or AWS API Gateway.

Residual risk. Medium for multi-instance deployments, low for single-instance.

Risk 5: TOCTOU Race on Custom Alias Creation

Description. WriteService.shorten() checks existsByCustomAlias() and then saves. There's a race window between the check and the save.

Impact. Two concurrent requests for the same alias could both pass the check, and one would then fail with DataIntegrityViolationException from the DB UNIQUE constraint.

Mitigation. The UNIQUE constraint is the authoritative guard. The application check exists purely for a friendlier response in the normal, non-racing case. A more robust version would catch DataIntegrityViolationException and translate it to 409, which would eliminate the user-visible impact of the race entirely.

Residual risk. Low, the race window is sub-millisecond in practice. This is the kind of bug that never shows up in dev but appears under load. I caught it in review, not in a failing test.

Risk 6: Silently Dead Integration Tests (found and fixed)

Description. UrlControllerIT.java, 4 integration tests, was never executed by mvn test. Surefire's default include pattern doesn't match the Failsafe-convention *IT.java naming, and no maven-failsafe-plugin was bound in pom.xml. The build reported success; the tests simply never ran.

Impact. A false sense of coverage. The documented "10 unit + 4 integration tests, all passing" claim was only 10 out of 14 true for any build run before this fix. That's a real quality-gate gap, the kind that erodes trust in a "tests pass" signal over time.

Fix applied. Bound maven-failsafe-plugin to the integration-test/verify goals in pom.xml. Verified: mvn verify now runs and passes all 14 tests. Full traceability is in AI_WORKFLOW.md.

Residual risk. None for this specific gap, it's closed. General lesson: a passing build isn't proof of coverage. Check what actually ran, not just the reported result.

Risk 7: Known Frontend Dependency CVEs, Deferred

Description. npm audit reports 4 CVEs: a moderate esbuild issue (dev server accepts requests from any origin, affecting vite <=6.4.2 transitively) and a moderate React Router open-redirect/SSR-hydration issue (react-router 6.0.0-7.17.0). Both fixes require npm audit fix --force, a breaking major-version bump (Vite 5 to 8, React Router 6 to 7).

Impact. The esbuild issue only affects the local dev server, not the production Nginx-served build, so it's low real exposure. The React Router issue affects <Link>/useNavigate open-redirect behavior. Exposure is low in this app since there's no user-controlled redirect target rendered through routing, but it's still a real CVE against a shipped dependency.

Decision. Deferred rather than force-upgraded blind. A --force major bump on both build tooling and the routing library needs route-by-route regression testing before it's safe to ship, and that's out of scope for a same-day fix. I'd rather not run it unreviewed just to clear an audit warning.

Residual risk. Low for the dev-only exposure, medium for the unpatched known CVE in a shipped dependency. Tracked here, not silently ignored. This is an explicit follow-up, not an oversight.

Trade-offs

Trade-off 1: 302 vs. 301 Redirect

	302 Found (chosen)	301 Moved Permanently
Browser caching	Not cached, every click hits the server	Cached permanently, only first click tracked
Analytics	Every click tracked	Untracked after first cache
Performance	Slightly slower (round trip per click)	Faster for repeat visitors
Ability to change destination	Immediate	Cannot change once cached
SEO	Does not transfer PageRank	Transfers PageRank

Decision. 302. Analytics completeness is required, and the ability to delete or update URLs requires that redirects not be permanently cached client-side.

Trade-off 2: Base62 Counter vs. Hash-Based Codes

	Base62 counter (chosen)	Hash-based (MD5/SHA truncated)
Collision risk	Zero (sequential, globally unique)	Non-zero with short prefixes
Predictability	Guessable (sequential)	Appears random
Code length	Shorter for equal collision safety	Longer to maintain acceptable odds
Custom alias support	Easy	Easy
Distributed generation	Needs Redis coordination	Fully stateless

Decision. Base62 counter. Zero collision risk matters more than unpredictability here. If enumeration resistance were required, a random suffix could be layered on top.

Trade-off 3: Redis Cache TTL vs. Data Freshness

Cache TTL is 24 hours.

Concern	Impact / Mitigation
URL deleted	Cache could serve a stale URL for up to 24h. Mitigated: <code>WriteService.delete()</code> explicitly evicts the Redis key.
URL expires	Cache could serve an expired URL for up to 24h. Mitigated: the cache is only populated with non-expired URLs, so after expiry, a miss hits the DB, gets checked, returns 410, and isn't repopulated.
Destination changed	Not supported, there's no update endpoint, so the cache is consistent by design.
Redis memory	Long-tail URLs consume memory for 24h. Mitigated with configurable LRU eviction.

Decision. 24 hours balances hit rate against staleness risk, and explicit eviction on delete prevents the worst-case stale-serve scenario.

Trade-off 4: Hibernate Auto-DDL vs. Liquibase/Flyway

	ddl-auto: update (chosen)	Liquibase/Flyway
Setup complexity	Zero	Moderate (migration files)
Schema evolution	Automatic but limited (adds columns, won't drop)	Full control, rollback support
Production safety	Unsafe (no rollback)	Safe, versioned, auditable
Team collaboration	Poor (no change history)	Excellent
Prototype suitability	Good	Overkill

Decision. ddl-auto: update for the prototype. A production deployment would use Flyway with committed migration scripts instead.

Personal Reflection

Looking back, the one thing I'd change first before any production deployment is a dead-letter queue for Kafka. It's probably a 20-line change: configure a DLQ topic, route failed consumer messages there instead of dropping them. I deprioritized it because the prototype's fail-open behavior is acceptable for now, but silently losing click events in a real system isn't something I'd be comfortable shipping.

The in-memory rate limiter is the other early fix I'd make. It's fine for a single instance, but the moment you scale to two, a client can double their allowed rate. I knew this going in, it's a documented trade-off, but it's exactly the kind of thing that bites you the first time you scale horizontally.

The dead integration tests (Risk 6) were the more instructive find during this build, not because the fix was hard, but because it's the failure mode that's easy to miss: a build that reports success while quietly running less than it claims to. Worth checking `mvn verify` output line by line against what the docs say should run, instead of just trusting a green build.